

Architecture Design Record 001 - SED

GRUPO 2

Indice

1 ADR001: [Elección de la Arquitectura Monolito + Capas Combinado con los Patrones de Diseño MVC + Clean Architecture]	1
1.1 Estado	1
1.2 Contexto	1
1.3 Decisión	2
1.3.1 Diferencias entre Arquitectura en Capas y Clean Architecture	2
1.3.2 Integración entre MVC + Clean Architecture + Arquitectura en Capas	2
1.3.3 Estructura de Capas	3
1.3.4 Sobre el Modelo en la Capa de Presentación y en la Capa de Dominio .	4
1.4 Consecuencias	5
1.4.1 Positivas	5
1.4.2 Negativas	5

1 ADR001: [Elección de la Arquitectura Monolito + Capas Combinado con los Patrones de Diseño MVC + Clean Architecture]

1.1 Estado

[Aceptado]

1.2 Contexto

Actualmente, la Facultad de Ingenierías en Informática y Sistemas (FIIS) realiza la evaluación de docentes mediante formularios en papel o en Teams. Este método es manual, consume mucho tiempo, es propenso a errores y dificulta la generación de reportes y análisis estadísticos.

Se requiere un sistema digital que permita:

- **Evaluaciones en línea seguras y confiables**
- **Almacenamiento organizado de resultados**
- **Generación automática de reportes y estadísticas**
- **Mayor productividad en la gestión académica**

1.3 Decisión

Se está proponiendo una arquitectura de Capas combinado con Monolito, debido principalmente al conocimiento de nuestro equipo en estas arquitecturas, lo que disminuirá los tiempos de desarrollo, además de que esta arquitectura se caracteriza por tener un corto tiempo de desarrollo y despliegue.

Además, se ha optado por combinar esta arquitectura con los patrones de diseño MVC y Clean Architecture para lograr una mejor organización de los componentes internos en cada una de las capas.

1.3.1 Diferencias entre Arquitectura en Capas y Clean Architecture

Arquitectura en capas	Clean Architecture
Las dependencias pueden ir hacia abajo (por ejemplo, UI conoce a los servicios)	Las dependencias solo pueden apuntar hacia el dominio (inversión de dependencias)
Orientada a la estructura de la app	Orientada al dominio y casos de uso
A veces las capas están fuertemente acopladas	Promueve bajo acoplamiento y alta cohesión

Se puede usar MVC en la capa de presentación dentro de una arquitectura más amplia basada en capas y/o Clean Architecture. Lo que haces es combinar niveles de abstracción, donde:

- MVC organiza la parte de presentación (UI).
- Clean Architecture organiza el sistema completo, con enfoque en el dominio y la independencia de dependencias.

1.3.2 Integración entre MVC + Clean Architecture + Arquitectura en Capas

Elemento	Función	Nivel
MVC	Organiza la interfaz y la interacción	Presentación

Elemento	Función	Nivel
Clean Architecture	Separa bien las responsabilidades y dependencias	Arquitectura general
Capas	Estructura la aplicación en niveles lógicos (UI, negocio, datos)	Organización

1.3.3 Estructura de Capas

Las capas tendran la siguiente division:

```
[ UI (Capa de presentación) ]
  View (V) - Vista del Usuario (HTML)
  Controlador (C) - Recibe requests del usuario
  Modelo (M) - Datos que se muestran (DTOs, ViewModels)

[ Capa de aplicación / casos de uso ]
  Orquesta la lógica de negocio
  Llama a servicios del dominio

[ Capa de dominio ]
  Entidades, lógica de negocio pura
  Interfaces (repositorios, servicios)

[ Capa de infraestructura ]
  Implementaciones reales: bases de datos, APIs externas, etc.
```

Cada capa haría lo siguiente:

1. *Capa de Presentación*: Muestra datos al usuario y recibe las acciones del usuario. No tiene lógica de negocio.
2. *Capa de Aplicación*: Orquesta casos de uso y no tiene lógica de negocio compleja, ademas de que usa interfaces del dominio.
3. *Capa de Dominio*: Contiene el “corazón” de la aplicación y no depende de ninguna tecnología. Contiene lógica pura y reglas de negocio. También contiene abstracciones e interfaces que establecen contratos con otras capas.
4. *Capa de Infraestructura*: Implementa repositorios, conectores, clientes externos. Puede depender de ORMs, SDKs, etc.

El flujo ideal seria el siguiente:

1. Vista (V) muestra una interfaz al usuario.

2. Controlador (C) recibe una acción del usuario (ej: clic en botón).
3. El controlador llama a un caso de uso en la capa de aplicación.
4. El caso de uso interactúa con el dominio (entidades, validaciones).
5. El dominio usa repositorios (interfaces) para acceder a datos.
6. Las implementaciones reales de esos repositorios están en la infraestructura.
7. El resultado sube de vuelta y se muestra en la Vista.

1.3.4 Sobre el Modelo en la Capa de Presentación y en la Capa de Dominio

Característica	Modelo en MVC (presentación)	Modelo de Dominio (Clean Architecture)
Pertenece a la capa	Presentación	Dominio (núcleo)
Tipo de objeto	DTO / ViewModel	Entidad / Agregado
Usa lógica de negocio	No	Sí
Usado para mostrar datos	Sí	No directamente
Acoplamiento	Bajo	Muy bajo (independiente)

Función del Modelo en MVC (en presentación)	Ejemplo
Representar los datos para la Vista	Un objeto ProductoViewModel con nombre, precio, imagen.
Recoger los datos ingresados por el usuario	Un formulario HTML que se convierte en un objeto para el backend.
Validaciones simples de formato o campos	Campos requeridos, longitud, formato (no lógica de negocio).
Mapear datos de entrada/salida del controlador	Convertir un DTO a una Entidad , o viceversa.

1. Usuario entra a /productos/crear
2. Vista (V): muestra formulario para crear producto.
3. Modelo (M): **ProductoViewModel** tiene campos como nombre, precio, descripción.
4. Controlador (C):
 - Recibe el formulario lleno con datos.
 - Valida el **ProductoViewModel**.
 - Convierte a un **DTO** que se manda al caso de uso **CrearProductoUseCase**.
5. Resultado se regresa al Controlador, que pasa un nuevo **ProductoViewModel** a la Vista.

1.4 Consecuencias

1.4.1 Positivas

- *Mantenibilidad:* Gracias al patrón de diseño Clean Architecture, debido a su uso de abstracciones, separación de responsabilidades y aislamiento de la lógica de negocio.
- *Desarrollo e Implementación más Sencilla y Rápida:* El equipo está familiarizado con la arquitectura en capas y el patrón de diseño MVC, por lo que el tiempo para comprender el flujo de trabajo se reduce.

1.4.2 Negativas

- *Baja Escalabilidad:* La arquitectura en capas no es muy conocida por permitir la escalabilidad horizontal.
- *Baja Flexibilidad:* Nuevos requisitos o funcionalidades podrían necesitar la adición de nuevas capas, lo cual aumentaría la complejidad del esquema actual de 4 capas.
- *Mayor Probabilidad de Errores con el Control de Versiones:* Esta arquitectura no permite que cada integrante trabaje de manera completamente independiente en su área o módulo, lo cual podría traer problemas en un futuro al momento de juntar los cambios.